

Documentação Arquitetural e Banco de Dados

Projeto: Evolutto

Autor: Allan Victor Rodrigues Souza

Stack Tecnológica: Angular (Frontend), Java / Spring Boot (Backend), PostgreSQL, Redis.

Padrão Arquitetural: MVC (Model-View-Controller) / Monólito Modular.

1. Visão Geral do Produto

O Evolutto é uma aplicação de gestão de hábitos gamificada. O diferencial competitivo do sistema é a expansão da gamificação tradicional para dinâmicas sociais (competições entre amigos) e um módulo exclusivo de controle parental, garantindo flexibilidade na validação e gestão de tarefas.

2. Arquitetura do Sistema

A aplicação adotará a arquitetura de **Monólito Modular** dividida em três fases de implantação, garantindo entregas de valor contínuas. O backend será estruturado estritamente utilizando o padrão **MVC**, assegurando uma separação clara entre a camada de apresentação (Controllers REST), regras de negócio (Services) e acesso a dados (Models/Repositories).

- **Frontend (Views / UI):** Desenvolvido em Angular. Consumirá a API REST e gerenciará o estado local das telas.
- **Backend (Controllers & Models):** Spring Boot (Java). Focado em escalabilidade transacional e segurança.
- **Persistência Relacional:** PostgreSQL para garantir consistência transacional (ACID).
- **Cache e Leaderboards:** Redis para gerenciar sessões temporárias e o ranqueamento de alta concorrência.

3. Regras de Negócio e Mecânicas

3.1. Sistema Core (Hábitos Bons e Lojinha)

A dificuldade do hábito define o multiplicador de recompensa. As recompensas da Lojinha são personalizadas e criadas pelo próprio usuário. A compra de um item debita as moedas do saldo do usuário.

3.2. Mecânica de Hábitos Ruins e Debuffs

A execução de um hábito ruim não subtrai saldo de moedas nem de XP. Ao invés disso, aplica um estado de debuff no usuário, reduzindo em **50% o ganho de XP nos próximos 10 hábitos concluídos**.

3.3. Sistema de Validação (Anti-Cheat)

- **Modo Solo:** Baseado no sistema de honra.
- **Competições Sociais:** Baseado no sistema de honra, mas com um Log de Auditoria Social visível no feed.
- **Controle Parental (Maker/Checker):** O status vai para `PENDING_APPROVAL`. O responsável revisa e altera para `COMPLETED` ou `REJECTED`.

3.4. Castigos (Módulo Parental)

Os pais podem aplicar o status `FROZEN` na lojinha da criança. Para remover, cadastram uma "Missão de Redenção". Apenas após a conclusão e aprovação dessa missão, a loja é descongelada.

4. Escopo de Entrega (Faseamento)

- **Fase 1 - Core:** Autenticação, CRUD de hábitos, cálculo de XP/Moedas, Lojinha e controle do Debuff de -50%.
- **Fase 2 - Social:** Amizades, Grupos, Leaderboards processados no Redis, log visual de amigos.
- **Fase 3 - Parental:** Fluxo `PENDING_APPROVAL`, contas supervisionadas, congelamento de loja e redenção.

5. Esquema de Banco de Dados (PostgreSQL)

A modelagem utiliza `UUID` para chaves primárias, garantindo escalabilidade, além do uso de `ENUMS`, `CONSTRAINTS` e `INDEXES`.

5.1. Tipos Customizados (ENUMs)

```
CREATE TYPE user_role AS ENUM ('STANDARD', 'PARENT', 'CHILD');
CREATE TYPE habit_type AS ENUM ('GOOD', 'BAD');
CREATE TYPE habit_difficulty AS ENUM ('EASY', 'MEDIUM', 'HARD');
CREATE TYPE log_status AS ENUM ('COMPLETED', 'PENDING_APPROVAL', 'REJECTED');
CREATE TYPE shop_status AS ENUM ('ACTIVE', 'FROZEN');
```

5.2. Tabelas Core (Usuários, Hábitos e Logs)

```
CREATE TABLE users (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  username VARCHAR(50) UNIQUE NOT NULL,  
  email VARCHAR(255) UNIQUE NOT NULL,  
  password_hash VARCHAR(255) NOT NULL,  
  role user_role DEFAULT 'STANDARD',  
  current_xp INT DEFAULT 0,  
  current_coins INT DEFAULT 0,  
  level INT DEFAULT 1,  
  debuff_counter INT DEFAULT 0 CHECK (debuff_counter >= 0),  
  shop_status shop_status DEFAULT 'ACTIVE',  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE habits (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  title VARCHAR(100) NOT NULL,  
  description TEXT,  
  type habit_type NOT NULL,  
  difficulty habit_difficulty NOT NULL,  
  is_active BOOLEAN DEFAULT TRUE,  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE habit_logs (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  habit_id UUID NOT NULL REFERENCES habits(id),  
  user_id UUID NOT NULL REFERENCES users(id),  
  status log_status DEFAULT 'COMPLETED',  
  xp_rewarded INT DEFAULT 0,  
  coins_rewarded INT DEFAULT 0,  
  validator_id UUID REFERENCES users(id),  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
  validated_at TIMESTAMP WITH TIME ZONE  
);
```

5.3. Tabelas da Lojinha

```
CREATE TABLE rewards (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  title VARCHAR(100) NOT NULL,  
  cost INT NOT NULL CHECK (cost > 0),  
  is_active BOOLEAN DEFAULT TRUE,  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE reward_redemptions (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  reward_id UUID NOT NULL REFERENCES rewards(id),  
  user_id UUID NOT NULL REFERENCES users(id),  
  cost_applied INT NOT NULL,  
  redeemed_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);
```

5.4. Tabelas do Domínio Social (Grupos)

```
CREATE TABLE groups (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  creator_id UUID NOT NULL REFERENCES users(id),  
  name VARCHAR(100) NOT NULL,  
  description TEXT,  
  is_private BOOLEAN DEFAULT FALSE,  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE group_members (  
  group_id UUID NOT NULL REFERENCES groups(id) ON DELETE CASCADE,  
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  joined_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
  PRIMARY KEY (group_id, user_id)  
);
```

5.5. Tabelas do Domínio Parental

```
CREATE TABLE parental_relationships (  
    parent_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
    child_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
    PRIMARY KEY (parent_id, child_id)  
);  
  
CREATE TABLE redemption_missions (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    parent_id UUID NOT NULL REFERENCES users(id),  
    child_id UUID NOT NULL REFERENCES users(id),  
    title VARCHAR(150) NOT NULL,  
    status log_status DEFAULT 'PENDING_APPROVAL',  
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
    resolved_at TIMESTAMP WITH TIME ZONE  
);
```

5.6. Índices Estratégicos (Performance)

```
-- Acelera o carregamento do histórico do usuário na tela principal  
CREATE INDEX idx_habit_logs_user_date ON habit_logs(user_id, created_at DESC);  
  
-- Fundamental para o fluxo de aprovação parental  
CREATE INDEX idx_habit_logs_status ON habit_logs(status) WHERE status =  
'PENDING_APPROVAL';  
  
-- Acelera a busca de membros para sincronizar o ranqueamento com o Redis  
CREATE INDEX idx_group_members_group ON group_members(group_id);
```